

LD5 Sparse Density Optimization and Cache Specification

Exact vectorized evaluation, bounded immutable caches, and retained reference semantics

mdstats development specification

2026-07-20

Contents

Status and scope	1
Scientific invariants	1
Public optimization options	2
Vectorized CIC aggregation	2
Chunked canonical scatter	2
Immutable canonical-support cache	3
Optimized production pipeline	3
Resource policy	3
Determinism and concurrency	4
Validation matrix	4
Performance gates	4
Borrowed methods and attribution	4
Completion criteria	4

Status and scope

This specification governs architecture gate **LD5** for `mdstats`. It begins from `mdstats 0.19.50a0`, where dense, local block-sparse, and automatic backend selection are complete for atomic occupancy, framework-vertex occupancy, and framework-edge arc length. **Implementation status: completed in `mdstats 0.19.51a0`.**

LD5 optimizes the already certified single-level sparse path. It does not change the scientific estimator, logical grid, canonical stencil, edge quadrature, HDR semantics, block ownership, renderer, or LD4 backend-selection policy. The LD1-A flat-node path remains available as the simple numerical oracle.

The first production optimization set is deliberately limited to:

1. vectorized, preallocated periodic CIC contribution generation;
2. chunked vectorized canonical-stencil pair generation;
3. deterministic stable reduction of target-node contributions;
4. bounded process-local caching of immutable canonical stencil supports;
5. reuse of cached support objects between Phase-B planning and Phase-C realization;
6. auditable optimization and cache metadata.

Compiled extensions, GPU kernels, multiprocessing, and multilevel adaptive grids are non-objectives of this gate.

Scientific invariants

For identical weighted samples and options, the optimized and reference paths must produce the same sorted active logical nodes and satisfy

$$\frac{\|\rho_{\text{opt}} - \rho_{\text{ref}}\|_1}{\|\rho_{\text{ref}}\|_1} \leq 2 \times 10^{-12},$$

$$\frac{\|\rho_{\text{opt}} - \rho_{\text{ref}}\|_{\infty}}{\max \rho_{\text{ref}}} \leq 5 \times 10^{-12}.$$

The final integral error is bounded by

$$|M_{\text{field}} - M_{\text{target}}| \leq 5 \times 10^{-13} \max(1, M_{\text{target}}).$$

For each requested HDR fraction, the threshold difference is at most $5 \times 10^{-12} \max(1, \rho_{\text{max}})$ and the achieved-mass-fraction difference is at most 5×10^{-13} .

Optimization must not alter:

- the LD4 selected backend or selection reason;
- exact Phase-B hard counts;
- block ordering, masks, or serialization order;
- deterministic output under repeated execution;
- explicit dense and explicit reference behavior.

Public optimization options

```
@dataclass(frozen=True)
class DensityOptimizationOptions:
    sparse_evaluation_mode: str = "optimized" # optimized | reference
    cache_stencil_supports: bool = True
    sparse_pair_chunk_size: int = 262_144
```

Constraints:

- `sparse_evaluation_mode` is optimized or reference;
- `sparse_pair_chunk_size` is a positive integer;
- reference routes through the retained LD1-A implementation;
- optimized is the production default after LD5;
- cache use changes performance only and never changes scientific metadata used by LD4 selection.

Atomic and framework density options contain one `optimization_options` record. The record is additive and does not modify existing constructor defaults other than the new production sparse execution engine.

Vectorized CIC aggregation

The reference path enumerates the eight offsets in Python and concatenates temporary vectors. The optimized path preallocates arrays of length at most $8N_s$ and fills those arrays in the same offset-major, sample-stable order:

$$(0, 0, 0), (0, 0, 1), \dots, (1, 1, 1).$$

Zero contributions are discarded without changing the relative order of positive contributions. Reduction uses a stable target-node ordering so contributions to each node are accumulated in their original deterministic order.

The exact output contract is `SparseCICNodeMasses3D`; the reference function remains public and unchanged.

Chunked canonical scatter

Let N_o be the occupied CIC-node count and N_k the canonical stencil-offset count. The kernel-pair count is

$$N_p = N_o N_k.$$

The optimized path generates pairs in contiguous offset-major chunks. A chunk of B_k stencil offsets produces at most $B_k N_o$ pairs. Flattening the broadcast arrays in C order preserves the normative order

```
stencil offset 0, all sources;
stencil offset 1, all sources;
...
```

The full target and contribution vectors remain bounded by the existing exact `max_kernel_pairs` and `max_workspace_bytes` checks. The chunk size bounds temporary coordinate arrays and Python-loop overhead; it does not change the final reduction order.

For bounded logical grids, the optimized reducer may use a temporary dense float64 accumulator with `numpy.bincount`. This path is permitted only when the logical grid contains at most 4,194,304 nodes, the kernel-pair count is at least one eighth of the logical node count, and the scratch array fits the caller's workspace limit. Larger or more weakly populated grids use stable sorting by target node followed by ordered segment reduction. The temporary dense reducer is an optimization scratch array, not a scientific field or retained backend representation.

The LD1-A tolerance, rather than bit identity, is normative for optimized-versus-reference floating values. Active logical indices must remain exactly equal.

Immutable canonical-support cache

Canonical stencil supports are immutable scientific objects. LD5 introduces a bounded process-local least-recently-used cache keyed by the exact tuple

```
(logical grid shape, float64 display-cell bytes,  
 float64 Gaussian-bandwidth bits, float64 tail-tolerance bits)
```

The cache is bounded by both entry count and retained NumPy-array bytes. The default limits are:

```
maximum entries: 16  
maximum retained array bytes: 256 MiB
```

The cache is protected by a reentrant lock. Cache insertion and eviction order never enters a scientific result. Returned arrays remain read-only.

A cache hit must still enforce the caller's current candidate-contribution and workspace limits using diagnostics stored in the cached support. A support built under permissive limits may therefore be rejected by a later stricter call.

Public cache controls are:

```
clear_density_optimization_caches()  
density_optimization_cache_info()
```

The information record reports hits, misses, insertions, evictions, current entries, and retained array bytes. Cache counters are operational diagnostics and are not part of serialized scientific identity.

Optimized production pipeline

For local-sparse realization:

```
PeriodicWeightedSamples3D  
-> optimized or reference CIC aggregation  
-> cached or uncached PeriodicGaussianStencilSupport  
-> optimized or reference sparse scatter  
-> unchanged PeriodicBlockScalarField3D packing
```

Phase-B planning uses the same cache accessor as realization. Thus atomic, framework-vertex, and framework-edge fields sharing kernel geometry can reuse one certified support. The exact Phase-B node set, block plan, and LD4 selector remain unchanged.

The final field metadata records:

```
sparse_evaluation_mode  
cic_implementation  
scatter_implementation  
pair_chunk_size  
stencil_cache_enabled  
stencil_cache_hit_for_realization
```

Cache hit or miss is operational provenance only; it must not affect equality, serialization of scientific arrays, or backend selection.

Resource policy

The existing exact hard limits remain authoritative:

- max_cic_contributions;
- max_kernel_pairs;
- max_workspace_bytes;
- max_planning_bytes;
- block and stored-value limits.

Optimized workspace estimates include preallocated CIC vectors, pair vectors, stable sort order, reduction output, and chunk coordinate arrays. No optimization may bypass Phase-B or Phase-C approval.

Cached array bytes are process-retained memory outside one scene's transactional peak estimate. They are independently bounded and can be released explicitly. Scientific preparation must remain possible with caching disabled.

Determinism and concurrency

Repeated optimized calls with fixed inputs must produce identical arrays. Concurrent cache access may change only hit/miss counters. Construction occurs outside the cache lock; insertion uses deterministic key equality, and duplicate concurrent builds may be discarded safely.

Parallel numerical accumulation is deferred because deterministic summation and memory accounting require a separate specification.

Validation matrix

Required tests include:

1. optimized CIC versus LD1-A CIC for orthogonal and skew cells;
2. optimized scatter versus LD1-A scatter for all LD1-A scientific fixtures;
3. exact active-node and block-array identity under repeated execution;
4. cache hit, miss, eviction, clear, and stricter-limit rejection;
5. cache-disabled equivalence;
6. Phase-B planning followed by realization reuses the support;
7. atomic, framework-vertex, and framework-edge optimized/reference equivalence;
8. unchanged LD4 backend selection records;
9. localized and delocalized benchmarks;
10. unchanged explicit dense results.

Performance gates

Benchmarks are run with cache warm-up separated from timed iterations. On at least one localized and one broad-support fixture:

- optimized uncached sparse evaluation must not be more than 10% slower than the reference median;
- at least one representative sparse fixture must show a median speedup of 1.25x or greater;
- a warm support-cache hit must reduce support-preparation median time by at least 5x;
- retained cache bytes must stay within the configured bound.

Performance measurements are reported as evidence, not used as platform-fragile unit tests. Numerical and resource gates remain mandatory on every platform.

Borrowed methods and attribution

The periodic CIC assignment continues to follow Hockney and Eastwood, *Computer Simulation Using Particles* (1988). The LRU cache organization uses standard least-recently-used memoization; the scientific cache key, bounded byte policy, limit revalidation, and density integration are project-specific. Stable sorting and vectorized reduction use NumPy primitives and do not introduce a new scientific algorithm.

Completion criteria

LD5 is complete when:

1. the optimized production sparse path is available for atomic and framework fields;

2. the LD1-A reference path remains selectable and tested;
3. all optimized outputs satisfy the numerical criteria;
4. support caches are bounded, clearable, thread-safe, and limit-aware;
5. LD4 selection semantics and counts are unchanged;
6. benchmark evidence covers localized and broad-support fields;
7. explicit dense output remains compatible with `mdstats 0.19.50a0`;
8. the architecture standard and public exports are updated.